

네트워크 1 - OSI Layer

남궁명수

[OSI 7 Layer]

OSI 7계층은 네트워크 통신 과정을 역할별로 7단계로 나눈 개념적 표준 모델이다.

나누는 이유는 네트워크 통신 전체를 하나로 구현하면 너무 복잡하기 때문이다.
각 계층이 자신의 역할만 담당하게 만들어서 설계, 개발, 장애 분석을 쉽게 한다.

계층	이름	주요 역할	대표 기술·프로토콜
7	Application	사용자 애플리케이션에 네트워크 기능 제공	HTTP, DNS, SMTP, SSH
6	Presentation	데이터 형식 변환, 암호화, 압축	TLS, UTF-8, JPEG
5	Session	통신 세션 생성, 유지, 종료	세션 관리
4	Transport	프로세스 간 통신, 신뢰성, 흐름 제어	TCP, UDP
3	Network	다른 네트워크까지 패킷 전달	IP, ICMP, 라우터
2	Data Link	같은 네트워크 안에서 프레임 전달	Ethernet, MAC, ARP, 스위치
1	Physical	비트 신호를 실제 매체로 전송	전기 신호, 광 신호, 케이블

[1계층: Physical Layer]

데이터를 0과 1의 물리적인 신호로 변환해서 전송한다.

예를 들어, 다음과 같은 것들이 해당한다.

- 랜 케이블
- 광 케이블
- 전기 신호
- 전파
- 커넥터
- 리피터와 허브

1계층은 데이터가 HTTP 요청인지, IP 주소가 무엇인지 모른다. 단순히 비트를 신호로 바꿔 전달한다.

[2계층: Data Link Layer]

같은 로컬 네트워크 안에서 장치 간 데이터를 전달한다.

여기서는 IP 주소가 아니라 MAC 주소를 이용한다. 데이터 단위는 Frame이다.

이더넷 프레임에는 대략 다음 정보가 들어간다.

[목적지 MAC] [출발지 MAC] [유형] [데이터] [오류 검사용 정보]

주요 장비는 스위치이다. 스위치는 MAC 주소 테이블을 보고 어느 포트에 프레임을 전달할지 결정한다.
중요한 점은 인터넷 목적지까지의 MAC 주소를 사용하는 것이 아니라는 것이다.

예를 들어 내 컴퓨터가 Google 서버에 접속한다면

- 목적지 IP = Google 서버 IP
- 목적지 MAC = 현재 로컬 네트워크의 게이트웨이 MAC

Google은 다른 네트워크에 있으므로, 내 컴퓨터는 프레임을 우선 공유기 또는 기본 게이트웨이로 보낸다.

[3계층: Network Layer]

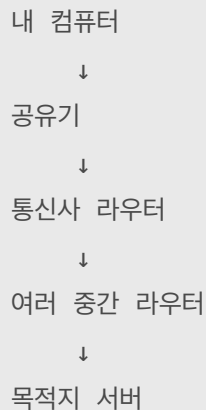
서로 다른 네트워크 사이에서 패킷을 전달한다.

대표 프로토콜은 IP이며, 주요 장비는 라우터다. 데이터 단위는 Packet이다.

IP 패킷에는 다음과 같은 정보가 포함된다.

- 출발지 IP(Internet Protocol)
- 목적지 IP
- TTL(Time To Live)
- 상위 프로토콜 번호
- TCP·UDP 등의 데이터

라우터는 목적지 IP 주소와 라우팅 테이블을 보고 다음으로 전달할 경로를 결정한다.



IP는 기본적으로 패킷을 전달하기 위해 노력하지만 다음을 보장하지 않는다.

- 패킷 도착
- 순서 보장
- 중복 방지
- 오류 복구

이러한 신뢰성은 TCP가 담당한다.

[4계층: Transport Layer]

호스트가 아니라 호스트 안에서 실행되는 특정 프로세스까지 데이터를 전달한다.

이를 위해 포트 번호를 사용한다.

IP 주소 → 어느 컴퓨터인가?
포트 번호 → 그 컴퓨터의 어느 프로세스인가?

예를 들어

142.251.118.113:80

- 142.251.118.114: 서버의 IP 주소
- 80: 서버에서 HTTP 서비스를 제공하는 포트

대표 프로토콜은 TCP와 UDP다.

[TCP]

연결 지향적이며 신뢰성 있는 통신을 제공한다.

- 3-way handshake를 통한 연결
- 데이터 순서 보장
- 손실된 데이터 재전송
- 중복 데이터 처리
- 흐름 제어
- 혼잡 제어

[UDP]

연결 과정과 재전송 기능이 없는 단순한 전송 프로토콜이다.

- 연결 설정 없음
- 도착 보장 없음
- 순서 보장 없음
- TCP보다 헤더와 처리 과정이 단순함
- 실시간 통신이나 간단한 질의에 적합

[5계층: Session Layer]

통신 세션을 생성하고 유지하며 종료하는 역할을 담당한다.

세션은 두 프로그램이 통신을 이어가기 위해 유지하는 논리적인 연결 상태라고 볼 수 있다.

다만 실제 인터넷 프로그램에서는 5, 6, 7계층이 명확하게 분리되어 구현되지 않는 경우가 많다.
TCP/IP 모델에서는 이 세 계층을 하나의 Application 계층으로 묶는다.

[6계층: Presentation Layer]

서로 다른 시스템이 데이터를 동일하게 해석할 수 있도록 표현 형식을 처리한다.

주요 역할은 다음과 같다.

- 문자 인코딩
- 직렬화와 역직렬화
- 압축과 압축 해제
- 암호화와 복호화

예를 들어, 서버가 JSON을 UTF-8 형식으로 전송하거나,
TLS(전송 계층 보안, Transport Layer Security)를 통해 데이터를 암호화하는 과정이 표현 계층의 역할로 설명될 수 있다.

[7계층: Application Layer]

사용자 프로그램과 가장 가까운 계층이다. 애플리케이션이 사용할 네트워크 통신 규칙을 정의한다.

대표적인 프로토콜은 다음과 같다.

- HTTP: 웹 데이터 전송
- DNS: 도메인 이름을 IP 주소로 변환
- SMTP: 이메일 전송
- SSH: 원격 접속
- FTP: 파일 전송

여기서 중요한 것은 Chrome이나 curl 자체가 HTTP는 아니라는 점이다.

- curl = HTTP 프로토콜을 사용하는 프로그램
- HTTP = 프로그램들이 따라야 하는 통신 규칙

[캡슐화와 역캡슐화]

송신 측에서는 위 계층에서 아래 계층으로 내려가며 각 계층의 헤더가 붙는다. 이를 캡슐화라고 한다.

HTTP 데이터

↓ TCP 헤더 추가

TCP 세그먼트

↓ IP 헤더 추가

IP 패킷

↓ Ethernet 헤더·트레일러 추가

Ethernet 프레임

↓

비트 신호

수신 측에서는 반대로 헤더를 하나씩 제거한다. 이를 역캡슐화라고 한다.

비트 신호

↓

Ethernet 프레임

↓ Ethernet 헤더 제거

IP 패킷

↓ IP 헤더 제거

TCP 세그먼트

↓ TCP 헤더 제거

HTTP 데이터

계층	데이터 단위
7~5계층	Data
4계층 TCP	Segment
4계층 UDP	Datagram
3계층	Packet
2계층	Frame
1계층	Bit

[TCP/IP 모델]

OSI는 네트워크를 설명하기 위한 이론적 모델이고,
TCP/IP는 실제 인터넷에서 사용되는 프로토콜 구조에 가깝다.

TCP/IP 계층	대응하는 OSI 계층	대표 프로토콜
Application	OSI 5~7계층	HTTP, DNS, SSH, TLS
Transport	OSI 4계층	TCP, UDP
Internet	OSI 3계층	IP, ICMP
Network Access	OSI 1~2계층	Ethernet, Wi-Fi, ARP

- L2 스위치: MAC 주소를 기반으로 프레임 전달
- L3 라우터: IP 주소를 기반으로 패킷 전달
- L4 로드밸런서: IP와 포트를 기반으로 전달
- L7 로드 밸런서: HTTP, URL, 헤더, 쿠키 등을 확인하여 전달

[curl <http://google.com>의 전체 흐름]

이 명령을 실행하면 대략 다음 과정이 일어난다.

1. DNS를 이용해 google.com의 IP 주소를 구한다.
2. 커널이 라우팅 테이블을 보고 패킷을 보낼 네트워크 인터페이스와 게이트를 결정한다.
3. ARP를 이용해 게이트웨이의 MAC 주소를 확인한다.
4. 클라이언트가 서버의 80번 포트에 TCP 연결을 시도한다.
5. TCP 3-way handshake가 수행된다.
6. curl이 HTTP 요청을 보낸다.
7. 데이터에 TCP, IP, Ethernet 헤더가 순서대로 붙는다.
8. 프레임이 게이트웨이와 여러 라우터를 거쳐 서버에 전달된다.
9. 서버가 HTTP 응답을 반환한다.
10. 통신이 끝나면 TCP 연결을 종료한다.

과정	관련 계층
HTTP 요청 생성	Application
TCP 연결과 신뢰성 처리	Transport
목적지 IP까지 라우팅	Internet
게이트웨이 MAC으로 프레임 전송	Network Access
실제 전기·무선 신호 전달	Physical

1. DNS로 구글 서버의 IP 주소 확인

- > google.com 이라는 도메인을 실제 통신에 필요한 IP 주소로 변환
- > DNS 결과가 캐시에 있다면 DNS 서버에 새로 요청하지 않을 수도 있다.

2. 커널이 라우팅 테이블 확인

- > 목적지 IP가 같은 로컬 네트워크에 있는지 확인
- > 외부 네트워크라면 사용할 네트워크 인터페이스와 기본 게이트웨이를 결정한다.

3. ARP로 다음 목적지의 MAC 주소 확인

- > 구글 서버의 MAC 주소가 아니라, 같은 네트워크에 있는 기본 게이트웨이의 MAC 주소를 알아낸다.
- > 이더넷 프레임은 우선 게이트웨이까지만 전달되기 때문이다.

4. 구글 서버의 80번 포트에 TCP 연결 시도

- > `http://` 이므로 기본 목적지 포트는 80번이다.
- > HTTPS라면 일반적으로 443번 포트를 사용하고, TLS 연결 과정도 추가된다.

5. TCP 3-way handshake 수행

- 클라이언트 -> 서버: SYN, Seq=x
- 서버 -> 클라이언트: SYN+ ACK, Seq=y, Ack=x+1
- 클라이언트 -> 서버: ACK, Ack=y+1
- > 이 과정을 통해 TCP 연결을 맺고 양쪽의 초기 시퀀스 번호를 확인한다.

6. curl이 HTTP 요청 생성

-> 이미 만들어진 TCP 연결을 통해 다음과 같은 HTTP 메시지를 보낸다.

```
GET / HTTP/1.1
Host: google.com
```

7. HTTP 요청 데이터가 캡슐화되어 전송

HTTP 메시지

- TCP 세그먼트
- IP 패킷
- Ethernet 프레임

DNS 요청과 TCP 핸드셰이크 패킷을 포함해 네트워크로 데이터를 보낼 때마다 발생

8. 프레임과 패킷이 서버까지 전달

- 이더넷 프레임은 우선 게이트웨이로 전달
- 라우터들은 IP 헤더의 목적지 IP를 보고 다음 경로로 패킷을 넘긴다.

- 목적지 IP는 일반적으로 구글 서버 IP로 유지된다. 단, NAT가 있으면 일부 변경될 수 있다.
- MAC 주소는 라우터를 한 홉 지날 때마다 다음 구간에 맞게 바꾼다.
- 요청과 응답이 반드시 완전히 동일한 경로를 이용하는 것은 아니다.

9. 서버가 HTTP 응답 반환

- 서버도 응답 데이터를 HTTP → TCP → IP → Ethernet 순서로 캡슐화해 보낸다.
- 클라이언트에서는 반대로 역캡슐화하여 curl에 HTTP 응답을 전달한다.

10. TCP 연결 종료

- 더 이상 통신하지 않으면, 보통 FIN, ACK를 주고받는 TCP 연결 종료 과정을 거친다.
- 다만 HTTP Keep-Alive를 사용하면 연결을 즉시 닫지 않고 재사용할 수 있다.